

Криптография

Хеш-функции

Описание проблемы

Хеш-функции используются для обеспечения безопасности данных, например, для хранения паролей, создания цифровых подписей и защиты целостности данных.

Небезопасные хеш-функции, такие как MD5 или SHA-1, имеют уязвимости, которые могут быть использованы злоумышленниками.

Основные проблемы небезопасных хеш-функций:

1. **Коллизии:** Небезопасные хеш-функции подвержены коллизиям, когда два различных входных значения дают одинаковый хеш. Это позволяет злоумышленнику подменить данные, не изменив хеш-значение.
2. **Криптоанализ:** Устаревшие хеш-функции могут быть взломаны методами криптоанализа, такими как атаки по радужным таблицам или атаки перебором (bruteforce), что позволяет злоумышленнику восстановить оригинальные данные из хеша.
3. **Слабость к атаке "дней рождения":** Небезопасные хеш-функции более уязвимы к атакам "дней рождения", что уменьшает время и ресурсы, необходимые для нахождения коллизий.



Атака "дней рождения" — используемое в криптоанализе название для метода взлома шифров или поиска коллизий хеш-функций на основе [парадокса дней рождения](#). Суть метода состоит в значительном уменьшении количества передаваемых хеш-функции аргументов, необходимого для обнаружения коллизии.

Последствия эксплуатации

Использование небезопасных хеш-функций может привести к серьезным последствиям:

1. **Утечка данных:** Злоумышленники могут восстановить оригинальные данные из хеша, что приведет к компрометации конфиденциальной информации, такой как пароли или личные данные.
2. **Подмена данных:** Коллизии позволяют злоумышленникам подменить данные без изменения хеша, что может использоваться для подделки цифровых подписей или изменения целостности данных.

Как исправить?

Рекомендуется использовать современные криптографические хеш-функции, такие как SHA-3, BLAKE2. Эти алгоритмы разработаны для обеспечения высокой степени безопасности и устойчивости к атакам.

Хеширование паролей

Для хеширования паролей рекомендуется использовать адаптивные хеш-функции, такие как bcrypt, scrypt, Argon2 или PBKDF2. Эти функции специально разработаны для защиты паролей, используя механизмы увеличения сложности и ресурсозатратности вычислений.

Использование соли для хеширования паролей:

Для защиты паролей рекомендуется использовать соль (salt). Соль добавляется к паролю перед хешированием, что делает каждое хеш-значение уникальным и защищает от атак с использованием радужных таблиц.



Радужная таблица — это предвычисленный список хешей для множества возможных паролей, который позволяет быстро находить исходные пароли по их хешам.

Пусть пароли хешируются по алгоритму MD5 и хранятся в виде хеш-значений в базе данных. В случае кражи базы данных исходные пароли могут быть восстановлены с помощью заранее подготовленных радужных таблиц, так как пользователи часто используют ненадежные, легко подбираемые по словарям пароли.

Однако если добавить "соль" к паролю, то есть присоединить к паролю строку из случайных символов перед хешированием, то результирующие хеш-значения не будут совпадать с распространенными словарями хешей. Хотя

знание соли позволяет злоумышленникам создать новые радужные таблицы для перебора, использование соли с высокой энтропией и достаточной длиной значительно усложнит эту задачу.

Поэтому соль должна быть достаточно сложной и уникальной для каждого пароля.

```
hashlib.md5("hello") = 5d41402abc4b2a76b9719d911017c592
hashlib.md5("hello" + "QxLUF1bgIAdeQX") = 61819a2e9b721831243eba1aeaba8486
hashlib.md5("hello" + "bv5PehSMfV11Cd") = 3c3fbfc82061e60fb6c11a5e0307058e
hashlib.md5("hello" + "YYLmfY6IehjZMQ") = 104f4f46d8222b57cbd543d75a3b8947
```

Пример использования соли при хешировании (для наглядности использован алгоритм MD5)



Соль не обязательно держать в секрете. Злоумышленник не узнает заранее, какая будет соль, поэтому он не может предварительно вычислить таблицу поиска или радужную таблицу.



Алгоритм действий злоумышленника при получении хеша пароля

1. Нет соли — используем **уже готовые** радужные таблицы
2. Одна соль на всех — генерируем **одну** радужную таблицу и «ломаем» по ней **всех** пользователей
3. Отдельная соль для каждого пользователя — **отдельно** «ломаем» каждого пользователя

Последний пункт требует больше всего времени и ресурсов злоумышленника



Современные алгоритмы хеширования, такие как Argon2id, bcrypt и PBKDF2, автоматически солят пароли, поэтому при их использовании не требуется никаких дополнительных действий.

Надежные пароли, хранящиеся с использованием современных алгоритмов и лучших практик хеширования, практически невозможно взломать злоумышленнику. Как разработчик, вы несете ответственность за выбор современного алгоритма хеширования.

Алгоритмы шифрования

Описание проблемы

Алгоритмы шифрования используются для обеспечения конфиденциальности, целостности и аутентичности данных. Они применяются в различных областях и сценариях, включая передачу данных, хранение информации и проверку подлинности.

Основные проблемы небезопасных алгоритмов шифрования:

1. **Устаревшие алгоритмы:** Алгоритмы, такие как DES (Data Encryption Standard), имеют недостаточную длину ключа и могут быть взломаны с использованием современных методов атаки.
2. **Алгоритмы с уязвимостями:** Например, RC4 (Rivest Cipher 4) и Triple DES (3DES) имеют известные уязвимости, которые позволяют злоумышленникам расшифровать данные.
3. **Слабые ключи:** Использование коротких или предсказуемых ключей делает шифрование неэффективным, так как злоумышленники могут легко их подобрать.
4. **Надежность хранения ключей:** Ключи должны быть защищены от несанкционированного доступа; неправильное хранение ключей приводит к компрометации всей системы шифрования.

Последствия эксплуатации

Использование небезопасных или устаревших алгоритмов шифрования может привести к компрометации конфиденциальных данных.

Как исправить?

Использовать современные алгоритмы шифрования

- Для **симметричного** шифрования в качестве предпочтительного алгоритма следует использовать AES с ключом длиной не менее **128 бит** (в идеале **256 бит**) и безопасным режимом.

- Для **асимметричного** шифрования используйте криптографию на основе эллиптических кривых (ECC) с безопасной кривой, например Curve25519, в качестве предпочтительного алгоритма. Если криптография на основе эллиптических кривых недоступна - необходимо использовать RSA, убедитесь, что длина ключа составляет не менее **2048 бит**.

Существует множество других симметричных и асимметричных алгоритмов, которые имеют свои плюсы и минусы и могут быть лучше или хуже, чем AES, RSA или Curve25519 в конкретных случаях использования. При их рассмотрении следует учитывать ряд факторов, в том числе:

- Размер ключа
- Известные атаки и слабые места алгоритма
- "Зрелость" алгоритма
- Одобрение авторитетных организаций, таких как программа проверки криптографических алгоритмов NIST
- Производительность (как для шифрования, так и для дешифрования)
- Качество имеющихся библиотек
- Распространенность поддержки алгоритма

Безопасная генерация случайных чисел

Случайные числа (или строки) необходимы для различных критически важных функций безопасности, таких как генерация ключей шифрования, вектора инициализации, идентификаторов сеансов, CSRF токенов или токенов сброса пароля. Важно, чтобы они генерировались безопасно и чтобы злоумышленник не мог их угадать и предсказать.



Компьютеры, как правило, не могут генерировать по настоящему случайные числа (без специального оборудования), поэтому большинство систем и языков предоставляют два разных типа случайности.

- **Генераторы псевдослучайных чисел (ГПСЧ)** обеспечивают низкокачественную случайность, работают значительно быстрее и могут быть полезны для задач, не связанных с безопасностью, таких как упорядочивание результатов на странице или рандомизация элементов пользовательского интерфейса. Однако их нельзя использовать для критически важных с точки зрения безопасности задач, так как злоумышленники часто могут угадать или предсказать их результаты.
- **Криптографически стойкие генераторы псевдослучайных чисел (КСГПСЧ)** обеспечивают значительно более высокое качество случайности (большую энтропию), что делает их надежными для использования в функциях, критически важных для безопасности. Однако такие генераторы работают медленнее, требуют больше ресурсов ЦП и могут блокироваться при запросах больших объемов случайных данных. Поэтому для задач, не связанных с безопасностью, где требуется большое количество случайных чисел, их использование может быть нецелесообразным.

В таблице ниже показаны рекомендуемые функции для каждого языка, а также небезопасные функции, которые не следует использовать.

Язык	Небезопасные функции	Криптографически безопасные функции
C	random(), rand()	getrandom(2)
Java	Math.random() StrictMath.random() java.util.Random java.util.SplittableRandom java.util.concurrent.ThreadLocalRandom	java.security.SecureRandom java.util.UUID.randomUUID()
PHP	array_rand() lcg_value() mt_rand() rand() uniqid()	random_bytes() Random\Engine\Secure в PHP 8 random_int() в PHP 7 openssl_random_pseudo_bytes() в PHP 5
.NET/C#	Random()	RandomNumberGenerator
Objective-C	arc4random() arc4random_uniform()	SecRandomCopyBytes

	subclasses ofGKRandomSource rand() random()	
Python	random()	secrets()
Ruby	rand() Random	SecureRandom
Go	rand в пакете math/rand	пакет crypto.rand
Rust	rand::prng::XorShiftRng	rand::prng::chacha::ChaChaRng и другие библиотеки CSPRNGs
Node.js	Math.random()	crypto.randomBytes() crypto.randomInt() crypto.randomUUID()